

Project Title

Restaurant Management System

Project Submitted to

SRM University –AP ,Andhra Pradesh

For the partial fulfillment of the requirements to award the degree of

Bachelor of Technology in

Computer Science and Engineering

School of Engineering and Sciences

Submitted by

J. Rohit Venkata Manikanta (AP24110010952)



Under the Guidance of

Dr. Deepthi K C kakumani

SRM University –AP , Neerukonda,

Mangalagiri,Guntur

Andhra Pradesh-522240

[March-2026]

Table of Contents

S.No	Section
1.	Identification of Project Related to DBMS (Project Title)
2.	Project Background
3.	Description of the Project
4.	ER Diagram Creation
5.	Description of ER Diagram
6.	Conversion of ER Diagram into Tables
7.	Description of Tables
8.	Normalization of Tables up to 3-NF
9.	Creation of Data in the Tables
10.	SQL Queries (Subqueries, Aggregate Functions, Joins)
11.	Creation of Views using the Tables
12.	Conclusion of the Project

1. Identification of Project Related to DBMS (Project Title)

Project Title: Restaurant Management System

This project developed to manage important restaurant activities in an organized and efficient manner. This project is related to Database Management System because it stores, updates, retrieves, and manages structured data such as customer records, employee details, menu information, order details, table reservations, inventory stock, and payment records.

The system uses a relational database model where data is stored in separate tables with proper relationships using primary keys and foreign keys. This makes the data more consistent, secure, and easy to access. The main objective of the project is to reduce manual work and improve restaurant operations through a computerized database system.

2. Project Background

Restaurants handle many daily tasks like taking orders, maintaining menu details, tracking customers, managing reservations, checking inventory, and handling billing. Doing all this manually can cause delays and errors.

A Restaurant Management System helps automate these tasks using a database, making the work faster, more accurate, and easier to manage.

Restaurant Management System helps solve these problems by storing data in an organized database. It allows the restaurant to:

- maintain customer and employee details
- manage menu and food pricing
- record and track orders
- reserve tables in advance
- update stock automatically
- manage payments properly
- generate useful reports

3. Description of the Project

The Restaurant Management System is designed to automate restaurant operations using a relational database. It is useful for handling customer interactions, employee records, orders, billing, reservations, and stock control.

This system has three main user roles:

- **Admin:** Has full control over the system, including menu management, employee management, customer records, inventory, billing, and analytics.
- **Staff:** Can manage orders, update order status, handle reservations, update inventory, and manage billing-related tasks.
- **Customer:** Can view the menu, place orders, and check order history.

The project includes the following functional modules:

1.Customer Management

Stores customer details such as name, phone number, and preferences. This helps the restaurant serve customers better and maintain order history.

2.Employee Management

Stores information about employees such as employee name, role, and work shift. This helps in organizing the staff structure of the restaurant.

3.Menu Management

Stores menu item information including item name, price, category, and availability. Menu items can be added, updated, or removed based on restaurant needs.

4.Order Management

Stores orders placed by customers. Each order contains multiple items, and the total bill amount is calculated automatically.

5.Order Details Management

Since one order can contain many menu items, a separate table is used to store item-wise order information such as quantity and price.

6.Reservation Management

Allows customers to reserve tables in advance. This helps the restaurant manage seating and avoid crowding.

7.Inventory Management

Stores stock quantity of restaurant items. Inventory gets updated whenever orders are placed, helping the restaurant track available stock.

8.Payment and Billing Management

Stores billing details such as payment method, payment amount, and payment status. This helps in maintaining financial records.

9.Sales Analytics

Useful reports such as daily sales and weekly sales can be generated from payment and order records.

Overall, the system provides a complete database solution for restaurant management and improves speed, accuracy, and efficiency.

4. ER Diagram Creation

The ER diagram represents the entities of the Restaurant Management System and the relationships among them.

The ER diagram for this project can be created using online tools such as:

-> **dbdiagram.io**

The main entities in this project are:

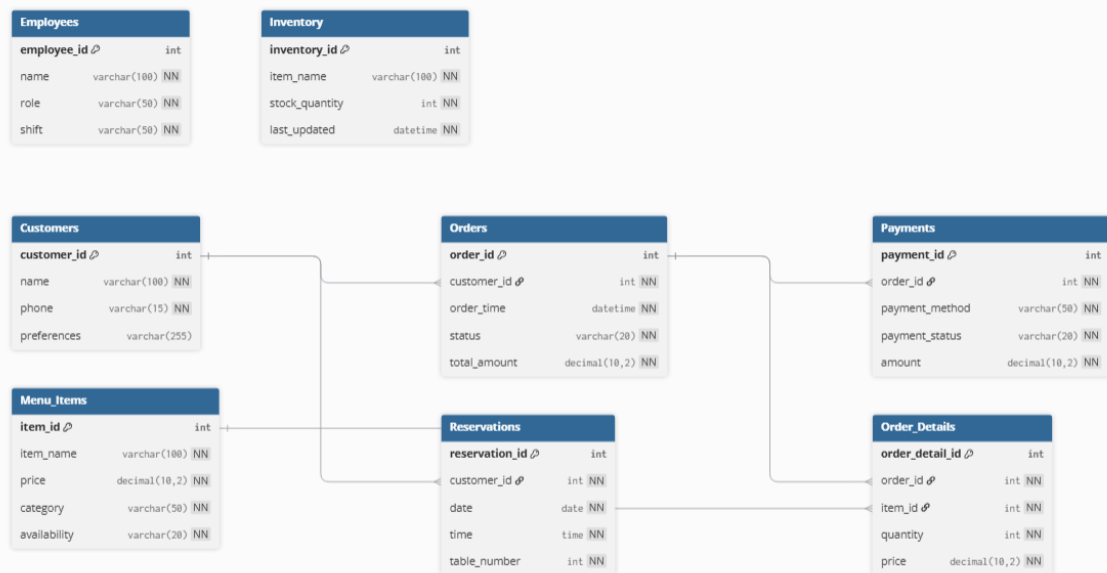
1. Customers
2. Employees
3. Menu_Items
4. Orders
5. Order_Details
6. Reservations
7. Inventory
8. Payments

The important relationships are:

- One customer can place many orders
- One customer can make many reservations
- One order can contain many order details

- One menu item can be part of many order details
- One order has one payment

You can draw the ER diagram using rectangles for entities, diamonds for relationships, and lines to show connections. Primary keys and foreign keys should also be marked clearly.



5. Description of ER Diagram

The ER diagram of the Restaurant Management System explains how different entities are connected with each other.

1. Customers Entity

This entity stores customer details. Each customer has a unique customer ID. A customer can place multiple orders and can also make multiple reservations.

2. Employees Entity

This entity stores employee information such as name, role, and shift. It helps the restaurant maintain records of its staff members.

3. Menu_Items Entity

This entity stores the food items available in the restaurant menu. Each item has a unique item ID, price, category, and availability status.

4. Orders Entity

This entity stores the order placed by customers. Each order belongs to one customer and has attributes like order time, status, and total amount.

5. Order_Details Entity

This entity stores detailed information about the items included in each order. It acts as a bridge between Orders and Menu_Items. One order can have multiple order details, and one menu item can appear in multiple order details.

6. Reservations Entity

This entity stores reservation details of customers, such as date, time, and table number. It is linked with the Customers entity.

7. Inventory Entity

This entity stores stock details of the food items or materials available in the restaurant. It helps in monitoring available quantity.

8. Payments Entity

This entity stores payment information for orders. Each payment is linked to one order and stores payment method, amount, and payment status.

Thus, the ER diagram clearly shows the logical structure of the project and how different entities interact with each other.

6. CONVERSION OF ER DIAGRAM INTO TABLES

Relational Schema

The entities identified in the ER diagram are converted into relational tables. Each entity becomes a table, and relationships are maintained using primary keys and foreign keys.

Tables and Their Structure

1. Customers Table

Customers(customer_id PRIMARY KEY, name, phone, preferences)

2. Employees Table

Employees(employee_id PRIMARY KEY, name, role, shift)

3. Menu_Items Table

Menu_Items(item_id PRIMARY KEY, item_name, price, category, availability)

4. Orders Table

Orders(order_id PRIMARY KEY, customer_id FOREIGN KEY, order_time, status, total_amount)

5. Order_Details Table

Order_Details(order_detail_id PRIMARY KEY, order_id FOREIGN KEY, item_id FOREIGN KEY, quantity, price)

6. Reservations Table

Reservations(reservation_id PRIMARY KEY, customer_id FOREIGN KEY, date, time, table_number)

7. Inventory Table

Inventory(inventory_id PRIMARY KEY, item_name, stock_quantity, last_updated)

8. Payments Table

Payments(payment_id PRIMARY KEY, order_id FOREIGN KEY, payment_method, payment_status, amount)

Table Creation

```
CREATE TABLE Customers (  
    customer_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    phone VARCHAR(15),  
    preferences VARCHAR(255)  
);
```

```
CREATE TABLE Employees (  
    employee_id INT PRIMARY KEY,  
    name VARCHAR(100),  
    role VARCHAR(50),  
    shift VARCHAR(50)  
);
```

```
CREATE TABLE Menu_Items (  
    item_id INT PRIMARY KEY,  
    item_name VARCHAR(100),  
    price DECIMAL(10,2),  
    category VARCHAR(50),  
    availability VARCHAR(20)  
);
```

```
CREATE TABLE Orders (  
    order_id INT PRIMARY KEY,
```

```
customer_id INT,  
order_time DATETIME,  
status VARCHAR(20),  
total_amount DECIMAL(10,2),  
FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);
```

```
CREATE TABLE Order_Details (  
    order_detail_id INT PRIMARY KEY,  
    order_id INT,  
    item_id INT,  
    quantity INT,  
    price DECIMAL(10,2),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),  
    FOREIGN KEY (item_id) REFERENCES Menu_Items(item_id)  
);
```

```
CREATE TABLE Reservations (  
    reservation_id INT PRIMARY KEY,  
    customer_id INT,  
    date DATE,  
    time TIME,  
    table_number INT,  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);
```

```
CREATE TABLE Inventory (  
    inventory_id INT PRIMARY KEY,  
    item_name VARCHAR(100),  
    stock_quantity INT,  
    last_updated DATETIME  
);
```

```
CREATE TABLE Payments (  
    payment_id INT PRIMARY KEY,  
    order_id INT,  
    payment_method VARCHAR(50),  
    payment_status VARCHAR(20),  
    amount DECIMAL(10,2),  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id)  
);
```

Screenshots:-

This document doesn't have any headings. To add headings to your Table of Contents, go to Home > Styles

```
Command Prompt - "C:\Progr  X  +  v

mysql> CREATE TABLE Order_Details (
->     order_detail_id INT PRIMARY KEY,
->     order_id INT,
->     item_id INT,
->     quantity INT,
->     price DECIMAL(10,2),
->     FOREIGN KEY (order_id) REFERENCES Orders(order_id),
->     FOREIGN KEY (item_id) REFERENCES Menu_Items(item_id)
-> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE Reservations (
->     reservation_id INT PRIMARY KEY,
->     customer_id INT,
->     date DATE,
->     time TIME,
->     table_number INT,
->     FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
-> );
Query OK, 0 rows affected (0.06 sec)

mysql> CREATE TABLE Inventory (
->     inventory_id INT PRIMARY KEY,
->     item_name VARCHAR(100),
->     stock_quantity INT,
->     last_updated DATETIME
-> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE Payments (
->     payment_id INT PRIMARY KEY,
->     order_id INT,
->     payment_method VARCHAR(50),
->     payment_status VARCHAR(20),
->     amount DECIMAL(10,2),
->     FOREIGN KEY (order_id) REFERENCES Orders(order_id)
-> );
Query OK, 0 rows affected (0.06 sec)
```

7.Description of Tables

1. Customers Table

This table stores information about customers.

Column Name	Data Type	Description
customer_id	Int (PK)	Unique id for each customer
name	Varchar(100)	Name of the customer
phone	Varchar(15)	Contact number of customer
preferences	Varchar(255)	Food preferences of customer

2. Employees Table

This table stores details of employees.

Column Name	Data Type	Description
employee_id	Int (PK)	Unique id for each employee
name	Varchar(100)	Name of the employee
role	Varchar(50)	Role of the employee
shift	Varchar(50)	Shift timing of employee

3. Menu_Items Table

This table stores information about menu items.

Column Name	Data Type	Description
item_id	Int (PK)	Unique id for each menu item
item_name	Varchar(100)	Name of the food item
price	Decimal(10,2)	Price of menu item
category	Varchar(50)	Category of food item
availability	Varchar(20)	Availability status of item

4. Orders Table

This table stores customer order details.

Column Name	Data Type	Description
order_id	Int (PK)	Unique id for each order
customer_id	Int (FK)	References the customer who placed the order
order_time	DateTime	Time when order was placed
status	Varchar(20)	Status of the order
total_amount	Decimal(10,2)	Total bill amount of order

5. Order_Details Table

This table stores item-wise details of each order.

Column Name	Data Type	Description
order_detail_id	Int (PK)	Unique id for each order detail
order_id	Int (FK)	References the related order
item_id	Int (FK)	References the related menu item
quantity	Int	Quantity of item ordered
price	Decimal(10,2)	Price of ordered item

6. Reservations Table

This table stores reservation details.

Column Name	Data Type	Description
reservation_id	Int (PK)	Unique id for each reservation
customer_id	Int (FK)	References the customer
date	Date	Reservation date
time	Time	Reservation time
table_number	Int	Reserved table number

7. Inventory Table

This table stores stock information.

Column Name	Data Type	Description
inventory_id	Int (PK)	Unique id for each inventory record
item_name	Varchar(100)	Name of stock item
stock_quantity	Int	Available stock quantity
last_updated	DateTime	Last updated date and time

8. Payments Table

This table stores payment details of orders.

Column Name	Data Type	Description
payment_id	Int (PK)	Unique id for each payment
order_id	Int (FK)	References the related order
payment_method	Varchar(50)	Method of payment
payment_status	Varchar(20)	Payment status
amount	Decimal(10,2)	Amount paid for the order

Conclusion

Each table is structured with appropriate primary keys and foreign keys to maintain relationships between data. This design ensures efficient storage, retrieval, and management of customer details, menu information, orders, reservations, inventory, and payment records in the Restaurant Management System.

8. Normalization of Tables up to 3-NF

Normalization is the process of arranging data in a database to eliminate redundancy and dependency problems. In this project, normalization is applied up to Third Normal Form.

First Normal Form (1NF)

A table is in 1NF if:

- all attributes contain atomic values
- there are no repeating groups
- each row is unique

In this project, each table stores only single-valued attributes. For example, the Customers table stores one phone number and one preference field per customer record. Similarly, each menu item, order, and payment record is stored in a separate row.

Second Normal Form (2NF)

A table is in 2NF if:

- it is in 1NF
- all non-key attributes are fully dependent on the primary key

In this project, all non-key attributes depend fully on their primary keys. For example:

- in Orders, order_time, status, and total_amount depend on order_id
- in Menu_Items, item_name, price, category, and availability depend on item_id
- in Order_Details, quantity and price depend on order_detail_id

Thus, there is no partial dependency.

Third Normal Form (3NF)

A table is in 3NF if:

- it is in 2NF
- there is no transitive dependency
- non-key attributes depend only on the primary key

In this project:

- customer information is stored only in the Customers table
- employee information is stored only in the Employees table
- menu information is stored only in the Menu_Items table
- payment details are stored only in the Payments table

For example, the Orders table stores customer_id but not customer name directly. Customer name is kept in the Customers table. This avoids transitive dependency and duplicate storage.

Therefore, the database design of the Restaurant Management System is normalized up to **3NF**, which improves consistency and reduces data redundancy.

9. Creation of Data in the Tables

The following sample records can be inserted into the tables to test the project.

1. Customers Table

INSERT INTO Customers (name, phone, preferences) VALUES

('Rahul Sharma', '9876543210', 'Less spicy'),
('Priya Singh', '9123456780', 'No onion'),
('Aman Verma', '9988776655', 'Extra cheese'),
('Sneha Kapoor', '9012345678', 'No garlic');

2. Employees Table

INSERT INTO Employees (name, role, shift) VALUES

('Rakesh Kumar', 'Admin', 'Full Day'),
('Sonia Patel', 'Staff', 'Morning'),
('Kunal Mehta', 'Chef', 'Evening'),
('Neha Joshi', 'Cashier', 'Morning');

3. Menu Table

INSERT INTO Menu_Items (item_name, price, category, availability) VALUES

('Paneer Tikka', 220.00, 'Starter', 'Available'),
('Tomato Soup', 120.00, 'Soup', 'Available'),
('Veg Biryani', 180.00, 'Main Course', 'Available'),
('Butter Naan', 40.00, 'Bread', 'Available'),
('Cold Coffee', 90.00, 'Beverage', 'Available'),
('Ice Cream', 80.00, 'Dessert', 'Available');

4. Orders Table

INSERT INTO Orders (customer_id, order_time, status, total_amount) VALUES

(1, '2026-04-11 12:30:00', 'Pending', 400.00),
(2, '2026-04-11 13:15:00', 'Completed', 180.00),
(3, '2026-04-11 14:00:00', 'Served', 300.00);

5. Order_Details Table

```
INSERT INTO Order_Details (order_id, item_id, quantity, price) VALUES
```

```
(1, 1, 1, 220.00),
```

```
(1, 5, 2, 90.00),
```

```
(2, 3, 1, 180.00),
```

```
(3, 2, 1, 120.00),
```

```
(3, 4, 2, 40.00),
```

```
(3, 6, 1, 80.00);
```

6.Reservations Table

```
INSERT INTO Reservations (customer_id, date, time, table_number) VALUES
```

```
(1, '2026-04-12', '19:00', 5),
```

```
(2, '2026-04-12', '20:00', 3),
```

```
(4, '2026-04-13', '18:30', 2);
```

7.Inventory Table

```
INSERT INTO Inventory (item_name, stock_quantity, last_updated) VALUES
```

```
('Paneer Tikka', 25, '2026-04-11 10:00:00'),
```

```
('Tomato Soup', 30, '2026-04-11 10:00:00'),
```

```
('Veg Biryani', 40, '2026-04-11 10:00:00'),
```

```
('Butter Naan', 100, '2026-04-11 10:00:00'),
```

```
('Cold Coffee', 30, '2026-04-11 10:00:00'),
```

```
('Ice Cream', 20, '2026-04-11 10:00:00');
```

8.Payments Table

```
INSERT INTO Payments (order_id, payment_method, payment_status, amount) VALUES
```

```
(1, 'Cash', 'Pending', 400.00),
```

```
(2, 'Card', 'Paid', 180.00),
```

```
(3, 'UPI', 'Paid', 300.00);
```

Screenshots:

```
C:\WINDOWS\system32\cmd. x + v
mysql> ^C
mysql> INSERT INTO Customers (name, phone, preferences)
      -> VALUES ('Rahul Sharma', '9876543210', 'Less spicy');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Employees (name, role, shift)
      -> VALUES ('Rakesh Kumar', 'Admin', 'Full Day');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Menu_Items (item_name, price, category, availability)
      -> VALUES ('Paneer Tikka', 220.00, 'Starter', 'Available');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Orders (customer_id, order_time, status, total_amount)
      -> VALUES (1, '2026-04-11 12:30:00', 'Pending', 220.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Order_Details (order_id, item_id, quantity, price)
      -> VALUES (1, 1, 1, 220.00);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Reservations (customer_id, date, time, table_number)
      -> VALUES (1, '2026-04-12', '19:00:00', 5);
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Inventory (item_name, stock_quantity, last_updated)
      -> VALUES ('Paneer Tikka', 25, '2026-04-11 10:00:00');
Query OK, 1 row affected (0.01 sec)

mysql>
mysql> INSERT INTO Payments (order_id, payment_method, payment_status, amount)
      -> VALUES (1, 'Cash', 'Pending', 220.00);
Query OK, 1 row affected (0.01 sec)

mysql>
```

10.SQL Queries (Subqueries, Aggregate Functions, Joins) on the Created Tables

The following SQL queries are used to retrieve useful information from the database.

1. Display all orders with customer names

```
SELECT o.order_id, c.name AS customer_name, o.order_time, o.status, o.total_amount  
FROM Orders o  
JOIN Customers c ON o.customer_id = c.customer_id;
```

2. Display order details with item names

```
SELECT o.order_id, c.name AS customer_name, m.item_name, od.quantity, od.price  
FROM Order_Details od  
JOIN Orders o ON od.order_id = o.order_id  
JOIN Customers c ON o.customer_id = c.customer_id  
JOIN Menu_Items m ON od.item_id = m.item_id;
```

3. Find total sales from paid payments

```
SELECT SUM(amount) AS total_sales  
FROM Payments  
WHERE payment_status = 'Paid';
```

4. Count total number of orders by status

```
SELECT status, COUNT(*) AS total_orders  
FROM Orders  
GROUP BY status;
```

5. Find average menu item price by category

```
SELECT category, AVG(price) AS average_price
FROM Menu_Items
GROUP BY category;
```

6. Find highest priced menu item

```
SELECT item_name, price
FROM Menu_Items
WHERE price = (
    SELECT MAX(price) FROM Menu_Items
);
```

7. Find customers who placed orders above 200

```
SELECT name
FROM Customers
WHERE customer_id IN ( SELECT customer_id FROM Orders WHERE
total_amount > 200 );
```

8. Display reservation details with customer names

```
SELECT r.reservation_id, c.name, r.date, r.time, r.table_number
FROM Reservations r
JOIN Customers c ON r.customer_id = c.customer_id;
```

9. Find total quantity sold for each menu item

```
SELECT m.item_name, SUM(od.quantity) AS total_quantity_sold
FROM Order_Details od
JOIN Menu_Items m ON od.item_id = m.item_id
GROUP BY m.item_name;
```

10. Find pending payments

```
SELECT p.payment_id, p.order_id, p.amount, p.payment_status
FROM Payments p
WHERE p.payment_status = 'Pending';
```

-

```

C:\WINDOWS\system32\cmd. X + v

mysql> SELECT o.order_id, c.name AS customer_name, o.order_time, o.status, o.total_amount
-> FROM Orders o
-> JOIN Customers c ON o.customer_id = c.customer_id;
+-----+-----+-----+-----+-----+
| order_id | customer_name | order_time      | status | total_amount |
+-----+-----+-----+-----+-----+
|          1 | Rahul Sharma  | 2026-04-11 12:30:00 | Pending |          220.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql>
mysql> SELECT o.order_id, c.name AS customer_name, m.item_name, od.quantity, od.price
-> FROM Order_Details od
-> JOIN Orders o ON od.order_id = o.order_id
-> JOIN Customers c ON o.customer_id = c.customer_id
-> JOIN Menu_Items m ON od.item_id = m.item_id;
+-----+-----+-----+-----+-----+
| order_id | customer_name | item_name      | quantity | price |
+-----+-----+-----+-----+-----+
|          1 | Rahul Sharma  | Paneer Tikka   |          1 | 220.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT SUM(amount) AS total_sales
-> FROM Payments
-> WHERE payment_status = 'Paid';
+-----+
| total_sales |
+-----+
|          NULL |
+-----+
1 row in set (0.00 sec)

mysql> ^C
mysql> SELECT status, COUNT(*) AS total_orders
-> FROM Orders
-> GROUP BY status;
+-----+-----+
| status | total_orders |
+-----+-----+

```

```
C:\WINDOWS\system32\cmd. X + v

mysql> ^C
mysql> SELECT status, COUNT(*) AS total_orders
      -> FROM Orders
      -> GROUP BY status;
+-----+-----+
| status | total_orders |
+-----+-----+
| Pending |          1 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT category, AVG(price) AS average_price
      -> FROM Menu_Items
      -> GROUP BY category;
+-----+-----+
| category | average_price |
+-----+-----+
| Starter |    220.000000 |
+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT item_name, price
      -> FROM Menu_Items
      -> WHERE price = (
      ->     SELECT MAX(price) FROM Menu_Items
      -> );
+-----+-----+
| item_name | price |
+-----+-----+
| Paneer Tikka | 220.00 |
+-----+-----+
1 row in set (0.00 sec)

mysql> SELECT name
      -> FROM Customers
      -> WHERE customer_id IN (
      ->     SELECT customer_id
      ->     FROM Orders
      ->     WHERE total_amount > 200
      -> );
```



```
C:\WINDOWS\system32\cmd. X + v

+-----+
| name   |
+-----+
| Rahul Sharma |
+-----+
1 row in set (0.01 sec)

mysql> SELECT name
      -> FROM Customers
      -> WHERE customer_id IN (
      ->     SELECT customer_id
      ->     FROM Orders
      ->     WHERE total_amount > 200
      -> );
+-----+
| name   |
+-----+
| Rahul Sharma |
+-----+
1 row in set (0.01 sec)

mysql> SELECT r.reservation_id, c.name, r.date, r.time, r.table_number
      -> FROM Reservations r
      -> JOIN Customers c ON r.customer_id = c.customer_id;
+-----+-----+-----+-----+-----+
| reservation_id | name       | date       | time       | table_number |
+-----+-----+-----+-----+-----+
| 1              | Rahul Sharma | 2026-04-12 | 19:00:00   | 5            |
+-----+-----+-----+-----+-----+
1 row in set (0.01 sec)

mysql> SELECT m.item_name, SUM(od.quantity) AS total_quantity_sold
      -> FROM Order_Details od
      -> JOIN Menu_Items m ON od.item_id = m.item_id
      -> GROUP BY m.item_name;
+-----+-----+
| item_name | total_quantity_sold |
+-----+-----+
| Paneer Tikka | 1 |
+-----+-----+
1 row in set (0.01 sec)
```

11. Creation of Views Using the Tables

Views are virtual tables created from one or more existing tables. They help simplify complex queries and present useful information in an easy format.

1. Customer Order View

```
CREATE VIEW Customer_Order_View AS
SELECT o.order_id, c.name AS customer_name, o.order_time, o.status,
o.total_amount
FROM Orders o JOIN Customers c ON o.customer_id = c.customer_id;
```

Purpose:

This view shows all orders along with the customer name, order time, order status, and total amount.

2. Billing View

```
CREATE VIEW Billing_View AS
SELECT p.payment_id, p.order_id, c.name AS customer_name,
p.payment_method, p.payment_status, p.amount
FROM Payments p
JOIN Orders o ON p.order_id = o.order_id
JOIN Customers c ON o.customer_id = c.customer_id;
```

Purpose:

This view is used to display billing and payment information for each order in a simple format.

3. Reservation View

```
CREATE VIEW Reservation_View AS
SELECT r.reservation_id, c.name AS customer_name, r.date, r.time,
r.table_number
FROM Reservations r JOIN Customers c ON r.customer_id =
c.customer_id;
```

Purpose:

This view shows reservation details along with the customer name and table number.

4. Sales Summary View

```
CREATE VIEW Sales_Summary_View AS
SELECT o.order_id, c.name AS customer_name, p.amount, p.payment_status
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id JOIN Payments p ON
o.order_id = p.order_id;
```

Purpose:

This view helps in checking sales records and payment status together.

12.Conclusion

The Restaurant Management System is a useful DBMS-based project developed to manage restaurant operations in an organized and efficient way. This project helps in storing and managing important data such as customer details, employee records, menu items, orders, reservations, inventory, and payments.

By using a relational database, the system reduces manual work, avoids data redundancy, and improves accuracy in maintaining records. The use of primary keys and foreign keys helps in maintaining proper relationships between tables. Normalization up to 3NF makes the database well-structured and efficient.

This project also demonstrates the practical use of SQL queries, joins, subqueries, aggregate functions, and views. Overall, the Restaurant Management System is a good example of how DBMS concepts can be applied in real-life applications to improve data handling, speed, and reliability.